Contents

1 Introduction……1
1.1 System Introduction……1
1.2 Background of the System……1
1.3 Objectives of the System……1
1.4 Significance of the System……1

2 Overall Description……2
2.1 Product Perspective……2
2.2 Product Scope……2
2.3 Product Functionality……2
2.4 Users and Characteristics……2
2.5 Operating Environment……2

3 Specific Requirements……3
3.1 Functional Requirements……3
3.2 Behaviour Requirements……3
3.3 External Interface Requirements……3

4 Other Non-functional Requirements……5
4.1 Performance Requirements……5
4.2 Safety and Security Requirements……5
4.3 Software Quality Attributes……5

5 Design Description……6
5.1 Composite Viewpoint……6
5.2 Logical Viewpoint……6
5.3 Information Viewpoint……6
5.4 Interaction Viewpoint……6
5.5 State Dynamics Viewpoint……6
5.6 Algorithm Viewpoint……6

1 Introduction

1.1 System Introduction

InterStock is a web-based Stock Market Learning Platform designed for academic use. It allows students to practice stocks, futures, and options trading through a real-time simulation using virtual money. The platform includes complete learning tools such as quizzes, assignments, study material, chatrooms, teacher monitoring, and AI-based performance analysis. The system supports both students and teachers, each having separate dashboards with different functionalities.

1.2 Background of the System

Most stock simulators only focus on simple virtual trading. They do not support classbased learning, teacher dashboards, progress tracking, or AI-driven performance evaluation. InterStock fills these gaps by offering real-time trading through APIs, teacher-controlled learning, assignments, quizzes, study material, chatrooms, and ML models.

1.3 Objectives of the System

InterStock aims to:

- **Provide Real-Time Trading Practice:** Offer students a practical stock market trading simulation using live market data for stocks, futures, and options.
- **Integrate Theory With Practice:** Combine theoretical learning through quizzes, notes, assignments, and study material with hands-on trading experience.
- **Support Teacher Management:** Enable teachers to upload and manage quizzes, assignments, announcements, and study materials through a dedicated dashboard.
- **Utilize ML for Evaluation:** Use machine learning models to analyze student trading behavior, predicting profit/loss and evaluating how well students understand trading concepts.
- **Introduce Ranking & Gamified Learning:** Implement rankings, achievements and Hall of Fame to make learning engaging and encourage consistent student participation.

1.4 Significance of the System

InterStock reduces the gap between finance theory and real-world market behavior.Students learn through practical trading, while teachers can monitor performance and teaching progress through dashboards.

The use of ML models enhances the system by predicting student profit/loss and understanding levels, making the platform more interactive and intelligent.

2 Overall Description

2.1 Product Perspective

InterStock is a web-based stock market learning system that interacts with several external services to deliver real-time financial education.

- **Frontend:** React.js is used to develop an interactive and responsive user interface for students and teachers.
- **Backend:** Node.js provides server-side logic, authentication, trading operations, and API communication.
- **Database:** MongoDB stores user accounts, portfolios, assignments, quizzes, chat messages, notes, and performance data.
- **Real-Time Trading Data:** Yahoo Finance and Alpha Vantage APIs supply live market prices and charts for stocks, futures, and basic options.
- **AI Models:** Machine learning models handle student trading prediction (profit/loss forecasting) and learning assessment (understanding vs memorization).
- **Communication:** Chatrooms and conversation modules support class discussions, private messaging, and teacher-monitored group chats.
- **User Roles:** The system supports two primary users Students and Teachers with separate dashboards and role-specific capabilities.

InterStock operates as an educational learning platform, integrating real-time market data, trading simulations, and classroom tools into a unified system. Unlike financial trading apps, InterStock focuses solely on academic use, combining theory with practical trading. It connects with external APIs for market data and provides a structured learning environment where students can trade, learn, communicate, and improve under teacher supervision.

2.2 Product Scope

InterStock focuses on real-time trading simulation, academic learning, and student–teacher interaction, excluding any form of real-money trading.

2.2.1 Included:

- **Student Dashboard:** Provides portfolio view, trading access and learning materials.
- **Teacher Dashboard:** Allows teachers to upload quizzes, assignments, study materials, and monitor student performance.
- **Real-Time Trading Simulation:** Supports virtual trading for stocks, futures, and basic options using live market data.

- **Assignments, Quizzes & Notifications:** Enables structured academic learning and timely reminders.
- **Study Material Upload:** Teachers can upload slides, notes, and learning resources.
- **Chatrooms:** Includes class-based chatrooms and private conversation modules for communication.
- **Notes Creation & Sharing:** Students can create personal notes and share them with the class.
- **Leaderboards & Hall of Fame:** Displays rankings and top-performing traders to promote competitive learning.
- **ML-Based Trading Prediction:** Predicts student profit/loss trends based on historical behavior.
- **ML-Based Learning Assessment:** Evaluates whether students understand concepts or rely on memorization.
- **Purchase History Tracking:** Stores virtual trading transactions.
- **User Profile Management:** Allows users to update personal details and account settings.

2.2.2 Excluded:

- **Real-Money Trading:** No actual financial transactions or real earnings.
- **Advanced Option Strategies:** Complex strategies (Iron Condor, Straddles, etc) are not included.
- **Real Brokerage Integration:** No linkage to live brokerage accounts or real trading platforms.
- **Financial Transactions:** No deposits, withdrawals, or payment processing of any kind.

2.3 Product Functionality

The platform's functionality is organized into major modules that collectively support student learning, teacher management, and real-time trading simulation.

2.3.1 User Authentication

- Student and Teacher login via Email or Google.
- Sign-up with unique email, one student can have only one active account.
- Forgot-password handled through email verification.
- Teachers have separate role-based access for managing academic content.
- Only the registered email/password can be used for future logins. ● Secure authentication using JWT.

2.3.2 Student Dashboard

The student dashboard consists of four primary sections:

A. Home Page

- Simulation portfolio view.
- Buy/Sell quick actions.
- Trending stocks.
- Quiz history display. Top navigation options:

1. Notifications
2. AI Chatbot
3. Menu dropdown containing:

- Assignments
- Quizzes
- Achievements
- Chatrooms
- Conversations
- Hall of Fame
- Notes
- Study Material
- Teachers
- Purchase History

2.3.3 Learn Section

Contains three modules:

1. Stocks (Ownership shares of a company whose value changes with its performance).

2. Futures (Contracts to buy or sell an asset at a fixed price on a future date).

3. Options (Contracts that give the right, but not the obligation, to buy or sell an asset).

Each module provides:

- Portfolio balance overview
- Buy/Sell trading actions
- Trending stocks
- Purchase history

2.3.4 Rank

Ranking based on:

- Quiz results
- Assignment scores
- Profit/Loss

- Hall of Fame

2.3.5 My Profile

Students can manage:

- Profile picture
- Notifications (ON/OFF)
- Profile details
- Password changes
- Personal notes
- Privacy policy & Terms
- Help center
- FAQs
- Contact form
- Account deactivation
- Logout

2.3.6 Teacher Dashboard

Teachers are provided tools to manage academic activities:

- Upload assignments
- Upload quizzes
- Upload study materials
- Create chatrooms
- Monitor class chat activity
- Check submissions
- Review student trading performance
- Send announcements
- Mark quizzes and assignments

2.3.7 Trading Module

- Real-time stock and futures price updates.
- Integration with Yahoo Finance & Alpha Vantage APIs.
- Buy/Sell operations.
- Interactive price graphs. ● Purchase history stored.

2.3.8 ML / AI Models

A. Trading Profit/Loss Prediction Model

- Trained on historical dataset.

- Predicts expected profit or loss for student trading behavior

B. Learning Assessment Model

- Analyzes quizzes, assignments, and trading patterns
- Determines whether the student has understood concepts or just memorized theory

2.3.9 **Notes Module**

- Students can create, edit, and delete personal notes
- Notes can be shared with the class
- Students can browse shared notes for collaborative learning

2.3.10 Chat System

- Teacher-monitored class chatrooms
- Private student-to-student conversations
- Student-created group chats
- Teacher–Student communication

2.3.11 Purchase History

- Displays all trading transactions
- Stores historical records

2.5 Users and Characteristics

1. Students:

- **Characteristics:** Primary users of the platform, mainly beginners who want to learn trading. No predefined levels, they progress by completing courses and practice activities.
- **Responsibilities:** Learn trading concepts, use the trading simulator, attempt quizzes and assignments, participate in discussions.
- **Usage Frequency:** Frequent, as they actively engage with learning materials and practice trading.
- **Technical Skill:** Basic (Students can use the platform and practice trading without needing to be tech experts or programmers).

2. Teachers:

- **Characteristics:** Educators or trainers responsible for guiding students. Familiar with trading concepts and online teaching tools.
- **Responsibilities:** Manage classes, upload learning materials, create evaluations, monitor student activity, track performance.

- **Usage Frequency:** Moderate to frequent, mainly during evaluation periods.
- **Technical Skill:** Basic to moderate (able to use dashboards, upload content, and manage evaluations).

2.6 Operating Environment

- **Frontend:** React.js for building a dynamic, interactive user interface, compatible with major browsers (Chrome, Edge, Firefox) on desktops, laptops, and tablets.
- **Backend:** Node.js for handling server-side logic, API requests, and communication with the database, running on Windows, Linux, or MacOS servers.
- **Database:** MongoDB for storing user data, trading history, quiz results, and other platform information·
- **APIs:** Alpha Vantage and Yahoo Finance APIs for fetching real-time
- **Client Requirements:** Stable internet connection, modern browser , minimum 4GB RAM for smooth UI performance.

3 Specific Requirements

3.1 Functional Requirements

This section details the functional requirements of the stock market learning platform, expanding on the general functionalities outlined in section 2.3. The requirements are organized into functional areas corresponding to the platform's core modules: User Authentication, Student & Teacher Dashboards, Trading, ML Models, Rankings, Profile Management, Assignments & Quizzes, Study Material, and Chat & Notes.

3.1.1 User Authentication :

This functional area handles secure user access and role-based account management.

- **User Registration/Login:** Users can register and log in using Email or Google accounts, with password reset functionality.
- **Role-Based Access:** The system assigns roles (Student, Teacher) to control access to specific features like Students access learning modules, Teachers manage classes.

- **JWT-Based Authentication:** The system uses JSON Web Tokens (JWT) to securely authenticate users after successful login. Each protected request from Students and Teachers requires a valid JWT to access system resources.

3.1.2 Student Dashboard

This functional area covers all student-facing activities.

- **Home Page:** Displays portfolio, buy/sell options,quiz history, notifications, chatbot, and navigation dropdown.
- **Learn Section:** Covers stocks, futures, and options, shows portfolio balance, trading options (buy/sell), charts and purchase history.
- **Rank Page:** Ranks students based on quiz scores, assignment performance, and trading P/L.
- **Assignments & Quizzes:** View and attempt assigned tasks.
- **Study Material:** Access slides and documents for learning.
- **Chatrooms:** Participate in class discussions.
- **Conversations:** Private chat with teachers or peers.
- **Notes:** Create and view personal and shared notes.

- **Purchase History:** View past transactions.

3.1.3 Teacher Dashboard

This area focuses on teacher functionalities.

- **Content Management:** Upload assignments, quizzes, and learning material.
- **Performance Monitoring:** Track student activity, progress, and trading performance.
- **Chatrooms Management:** Create and manage class chatrooms.
- **Announcements:** Send updates or notifications to students.
- **Submissions Checking:** Review submitted assignments and quizzes.

3.1.4 Trading Requirements

This functional area focuses on trading features for students and teachers.

- **Live Price Fetching:** Retrieve real-time market data via APIs.
- **Stock Charts:** Display interactive charts for stocks, futures, and options.
- **Profit/Loss Calculation:** Compute gains or losses automatically. ●**Transaction Storage:** Maintain a record of all trades.

3.1.5 ML Model Requirements

This area handles predictive and assessment models.

- **Profit/Loss Prediction:** ML models predict potential gains or losses for student trades.
- **Learning Assessment:** Evaluate student learning progress using performance metrics.

3.1.6 Rank Requirements

Ranks students based on performance metrics.

- **Scores:** Include quiz and assignment results.
- **Trading Performance:** Include trading P/L.
- **Overall Performance:** Combine scores and trading metrics for ranking.

3.1.7 Profile & Account Management

This area manages user account functionalities.

- **Profile Editing:** Update personal information, profile picture, and preferences.
- **Password Management:** Change or reset passwords.
- **Notifications Settings:** Enable or disable alerts for trades, quizzes, or announcements.

3.1.8 Assignments & Quizzes

Supports student learning evaluation.

- **View Tasks:** Students can see pending assignments and quizzes.
- **Attempt Tasks:** Complete quizzes or submit assignments online.

3.1.9 Study Material Management

- **Access Materials:** Students can view slides, documents, and resources uploaded by teachers.
- **Download Materials:** Option to download files for offline learning.

3.1.10 Chat & Notes

Supports communication and collaboration.

- **Chatrooms:** Class-wide discussions moderated by teachers.
- **Private Conversations:** One-on-one chats between students and teachers.
- **Notes:** Students can create personal or shared notes for study purposes.

3.2 Behavior Requirements

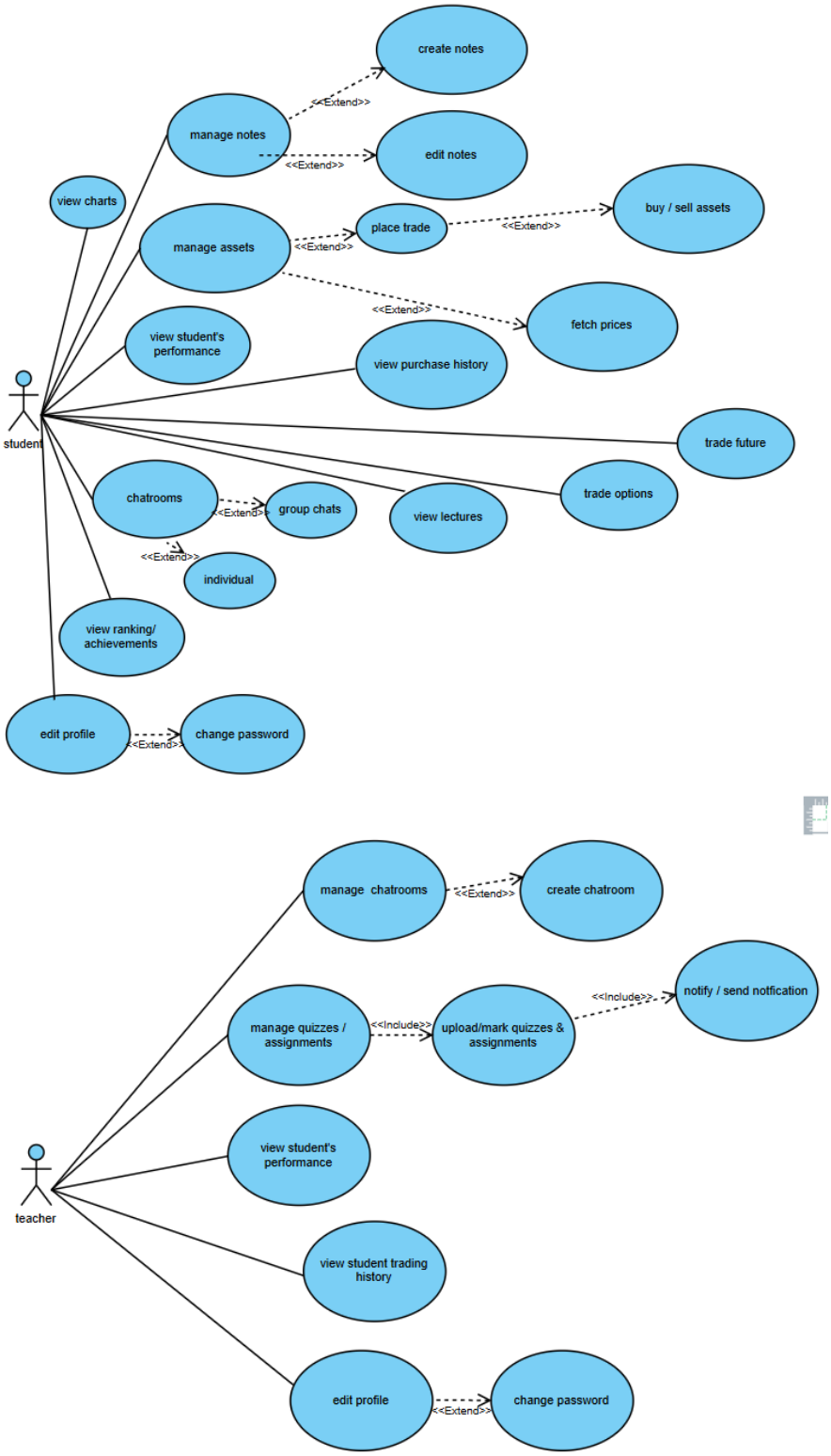


Figure 3.2.1 : Use Case Diagrams of Teacher & Student

3.3 External Interface Requirements

3.3.1 User Interfaces

1. Sign Up Page

Figure 3.3.1 shows the InterStock platform's "Create Account" page. It contains input fields for full name, email, confirm email, password, and confirm password.The interface includes a blue "Create Account" button for form submission.A link at the bottom allows existing users to navigate to the login page

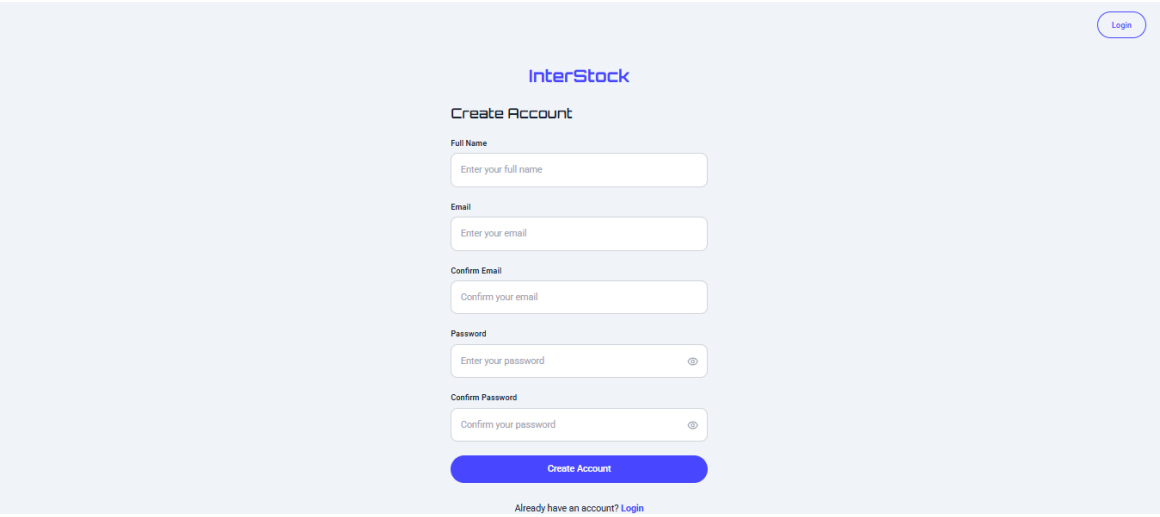


Figure 3.3.1 : User Account Registration Interface

2. **Login Page:**

Figure 3.3.2 illustrates the InterStock platform's login screen. It provides input fields for email and password, followed by a "Log into your Account" button. Users can also log in using Google through integrated social login options. A "Sign Up" link is available at the bottom for new users who need to create an account.

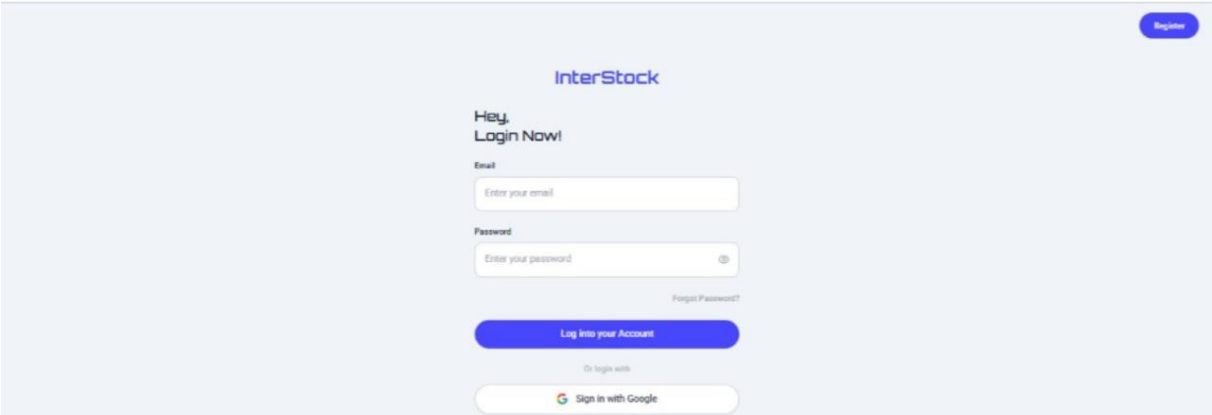


Figure 3.3.2 : User Login Interface

3. Home Page

Figure 3.3.3 displays the Home Page of the InterStock student dashboard.

The screen shows a real-time stock performance chart along with current portfolio value. A “Buy” and “Sell” button is visible below the chart for quick trading actions. Trending stocks such as AAPL, GOOG, and MSFT are listed on the side with their latest prices and percentage changes.

The Figure 3.3.4 shows a simulated stock trading interface for buying shares.The user can enter the stock symbol, quantity of shares, and see the price per share.The total amount is automatically calculated based on the quantity entered.Buttons are provided to either cancel the transaction or confirm the purchaseby clicking “Buy Stock.”A note at the bottom clarifies that this is a simulated environment, and no real money is used.

Figure 3.3.3: Student Dashboard Home Page

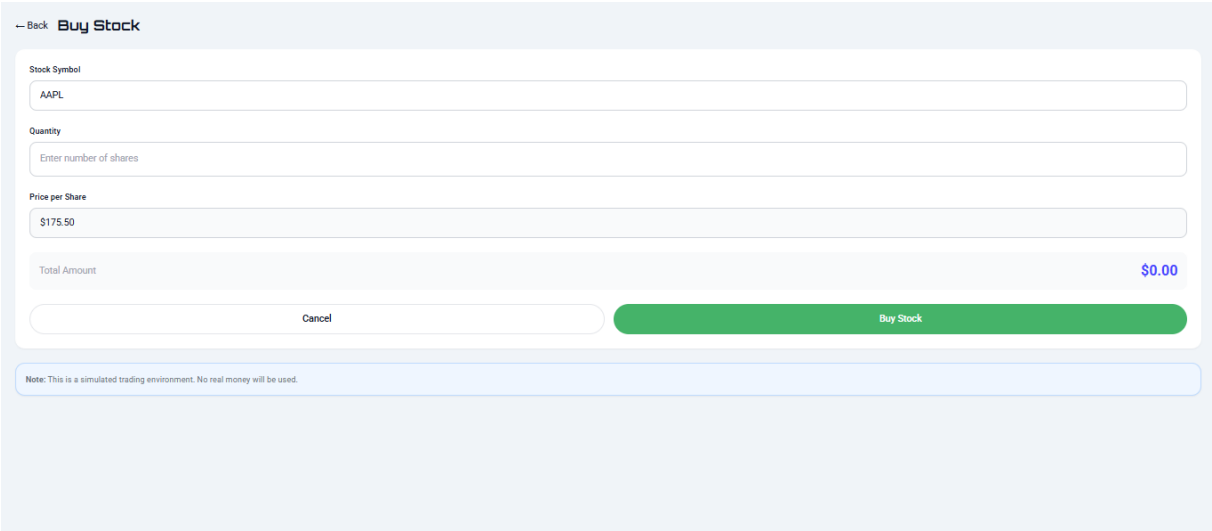


Figure 3.3.4: Buy Stock Interface.

4. Learn Trading Page:

This page shown in Figure 3.3.4 helps users learn about trading and the stock market. At the top, there is a search bar where users can type and find different stocks. Below that, there are

three tabs Stocks, Futures, and Options which give different ways to learn. The page also shows real-time stock prices like AAPL, GOOGL, MSFT, and TSLA, so users can see current market updates while learning.

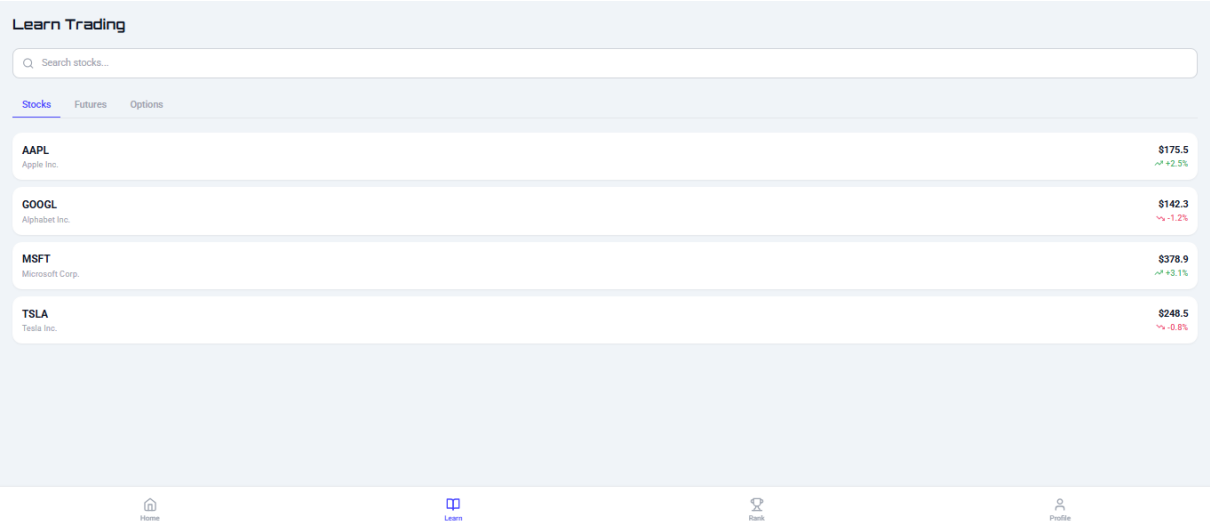


Figure 3.3.4 :Learn Trading Page

5. Rank Page

The Figure 3.3.5 shows a class ranking interface for students.Jawad is ranked #1 with a score of 95, followed by Maha (#2) with 92. The user ("You") is ranked #3 with a score of 88. Ayesha and Badar are ranked #4 and #5 with scores of 85 and 82, respectively.Icons for trophy, medal, and badges visually highlight the top performers.

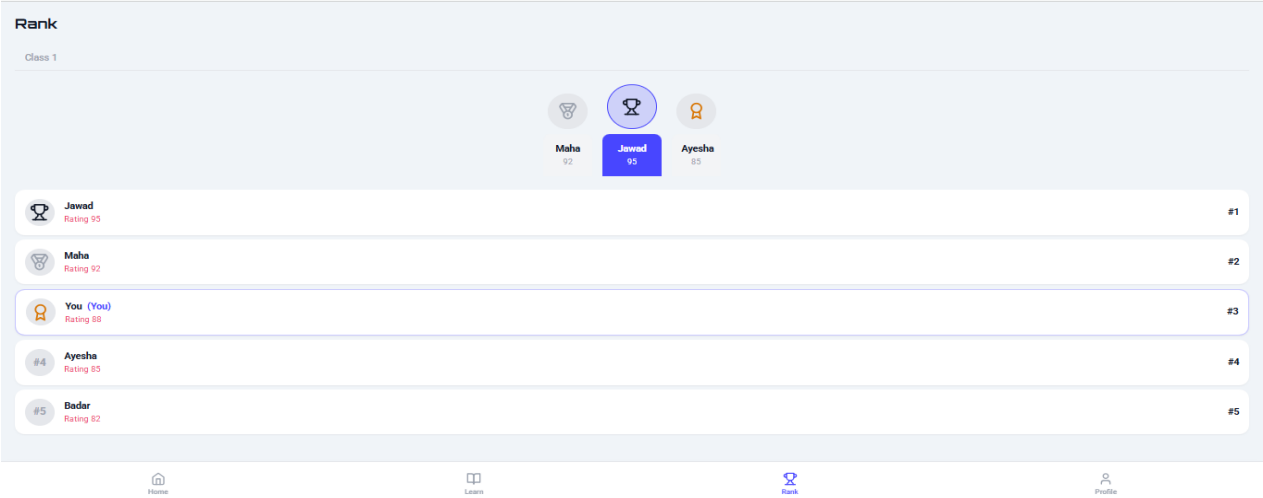


Figure 3.3.5 :Rank Leaderboard Page

6. Profile Page:

This page shown in Figure 3.3.6 displays the user profile for Ishmal Fatima. It allows the user to manage notifications, edit profile information, and change passwords. Other options include accessing notes, privacy policy, terms & conditions, and help resources. FAQs and contact support are also available for assistance. Navigation on

options at the bottom let the user switch between Home, Learn, Rank, and Profile sections.

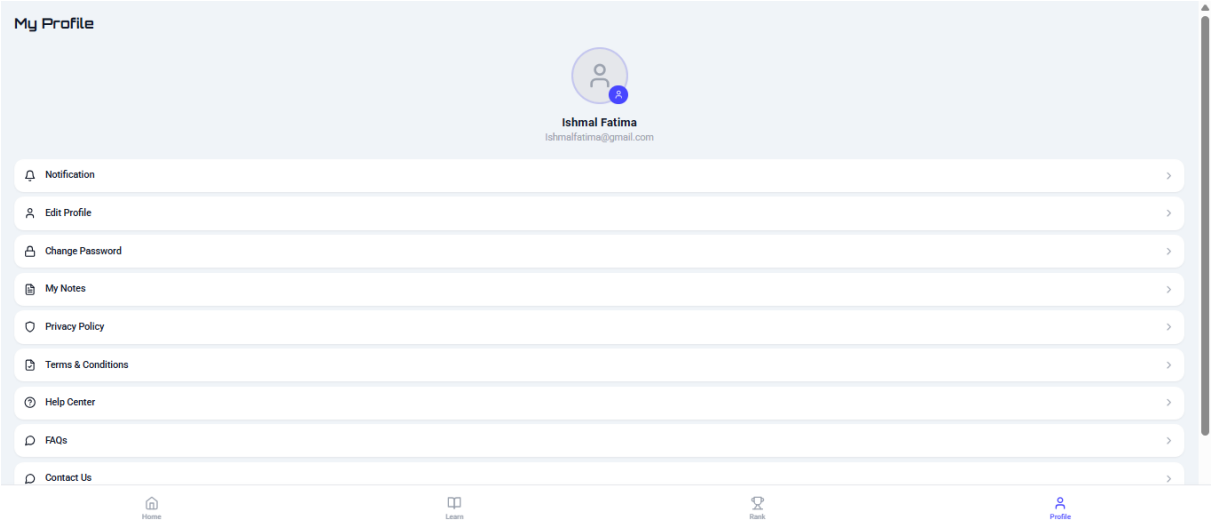


Figure 3.3.6 – User Profile Page.

7. **Edit Profile Page:**

This page shown in Figure 3.3.7 lets the user check and update their personal information. It shows fields like name, email, and phone number, which the user can edit. There is also a round area to add or change the profile photo. After making changes, the user can tap the "Save Changes" button at the bottom to update their details.

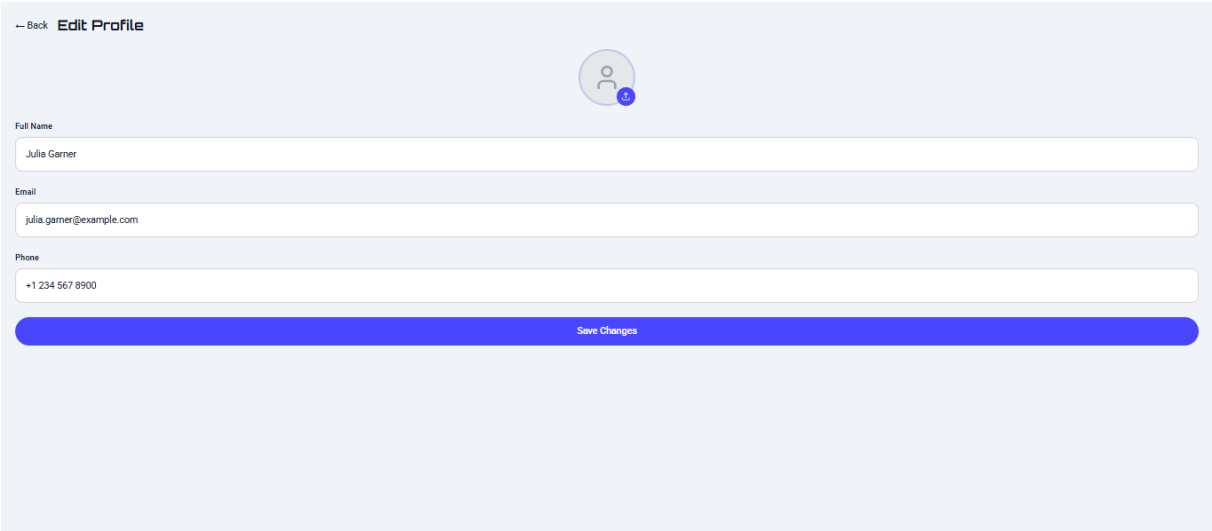


Figure 3.3.7 : Edit Profile Page

3.3.2 Hardware Interfaces

- The platform is fully web-based and requires no specialized hardware.
- Runs on standard client devices: PCs, laptops, and tablets.
- Compatible with modern browsers: Chrome, Edge, Firefox.

- No direct hardware communication is required; all functionality is handled via software interfaces.
- Trading simulator, dashboards, and chat features rely on browser APIs for input (e.g., mouse, keyboard, file uploads).
- Secure data exchange occurs over HTTPS/WebSocket; real-time updates do not require specialized hardware

3.3.3 Software Interfaces

Tools/Library /Version Relation

React.js	18.x	Frontend Framework
Node.js	22.x	Backend Runtime Environment
MongoDB	6.x	Database
Alpha Vantage API	Latest	Market Data Integration
Yahoo Finance API	Latest	Market Data Integration
Express.js	4.x	Backend Server Framework
Chart.js	Latest	Interactive Charts
JWT	Latest	Authentication
WebSocket	Latest	Real-time Communication

3.3.4 Communications Interfaces

- **HTTPS:** Secure client-server communication for login, trading data, profile updates, and API requests.
- **WebSocket:** Real-time updates for trading simulator, stock prices, and chat messages.
- **Data Security:** Sensitive data (passwords, transactions) encrypted
- **Performance:** Optimized for standard broadband connections (≥10 Mbps); asynchronous handling ensures smooth real-time updates.
- **Rate Limiting:** APIs limited to prevent abuse (e.g., 1000 requests/hour per user).

- **Optional WebRTC Support:** Could be used for real-time audio chat in chatrooms.

4 Other Non-functional Requirements

4.1 Performance Requirements

The platform's performance requirements ensure smooth operation of trading simulations, dashboards, quizzes, and real-time market data updates across standard devices and browsers.

4.1.1 Trading & Portfolio Performance:

- Real-time price updates from APIs shall refresh within 2 seconds.
- Portfolio balances, buy/sell orders, and profit/loss calculations shall update instantly upon trade execution.
- Charts shall render trading data in <1 second for standard timeframes (e.g., daily, weekly).

4.1.2 Dashboard & UI Performance:

- Student and Teacher dashboards shall load within 3 seconds on standard broadband.
- Navigation between sections (Learn, Rank, Profile, Chat) shall be seamless without page freezes.

4.1.3 API & Data Performance:

- API calls for stock/futures/options prices shall respond in <500ms under normal load
- System shall support at least 5,000 concurrent users without performance degradation.

4.1.4 Database & Scalability:

- MongoDB shall handle up to 100,000 student/teacher accounts with efficient query response times.
- Database scaling shall support increased data from trading transactions, assignments, and shared notes.

4.2 Safety and Security Requirements

4.2.1 Authentication & Access Control:

- Users must log in with secure credentials (email/password or Google).
- JWT (JSON Web Token) is issued upon login and required for accessing protected features, with server-side validation for each request.
- Role-based access ensures Students and Teachers access only authorized features.

- Automatic logout occurs after 30 minutes of inactivity.

4.2.2 Data Protection:

- Passwords shall be encrypted using encrypt.
- Sensitive data (trading history, notes, assignments) stored securely in MongoDB with access restrictions.
- File uploads (e.g., study material) are scanned to prevent malicious content.

4.2.3 API & Transaction Security:

- Trading APIs and portfolio updates use HTTPS with TLS 1.3 encryption.
- Rate limiting prevents abuse of API calls (e.g., 1000 requests/hour per user).
- Payment transactions, if any, are securely processed through PCI-compliant providers (Stripe, PayPal).

4.2.4 Backup & Recovery:

- Daily backups of MongoDB with 7-day retention.
- System shall recover from failure within 4 hours and data loss shall not exceed 15 minutes.

4.3 Software Quality Attributes

4.3.1 Usability:

- Provide a simple, modern, and intuitive UI for all users.
- Navigation between sections is clear and consistent; trading simulator is easy to use.
- Responsive design ensures usability on desktops, laptops, and tablets. ● Onboarding tutorials for new students and teachers.

4.3.2 Reliability:

- Ensure 99.9% uptime for dashboards, trading simulation, and quizzes.
- Failover mechanisms handle temporary API failures, fallback data used if live prices fail.
- Clear error messages for failed trades, invalid inputs, or system issues.

4.3.3 Scalability:

- Support growing user base and trading data without performance loss.
- MongoDB and APIs optimized to handle 100,000+ users and millions of transactions.
- Load balancing ensures even distribution of user requests.

4.3.4 Maintainability:

- Modular code structure (React.js frontend, Node.js backend) for easy updates.
- Comprehensive logging for debugging (API errors, failed trades, dashboard issues).

- Automated tests for critical modules (trading calculations, quizzes, portfolio updates).
- Well-documented code and APIs for future developers.

5 Design Description

5.1 Composite Viewpoint

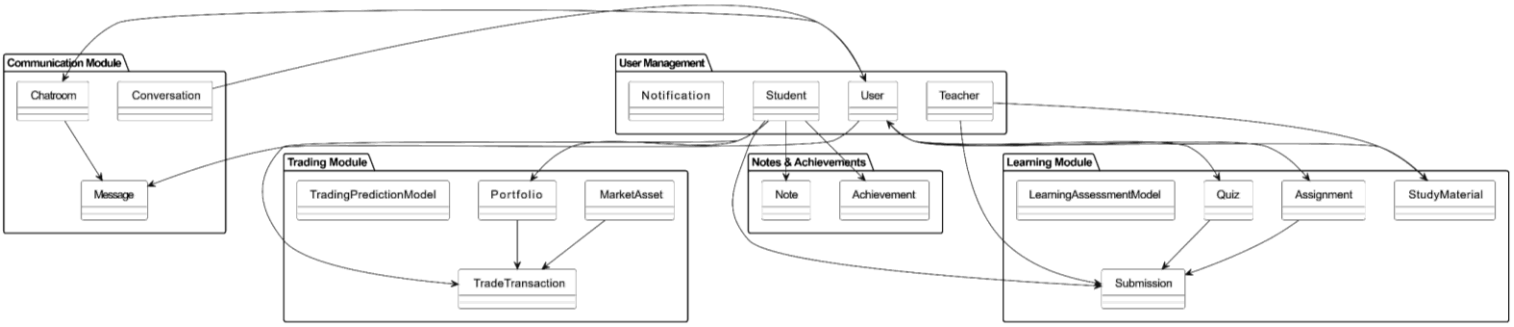


Figure 5.1.: Package Diagram

5.2 Logical Viewpoint

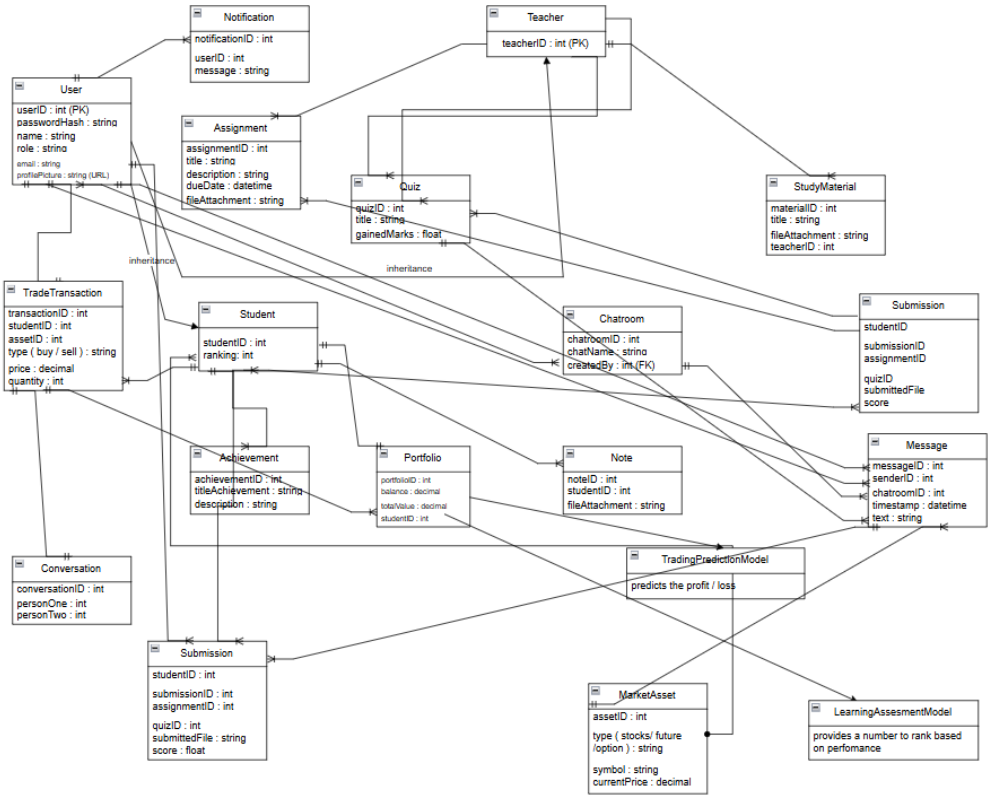


Figure 5.2.: Class Diagram

5.3 Information Viewpoint

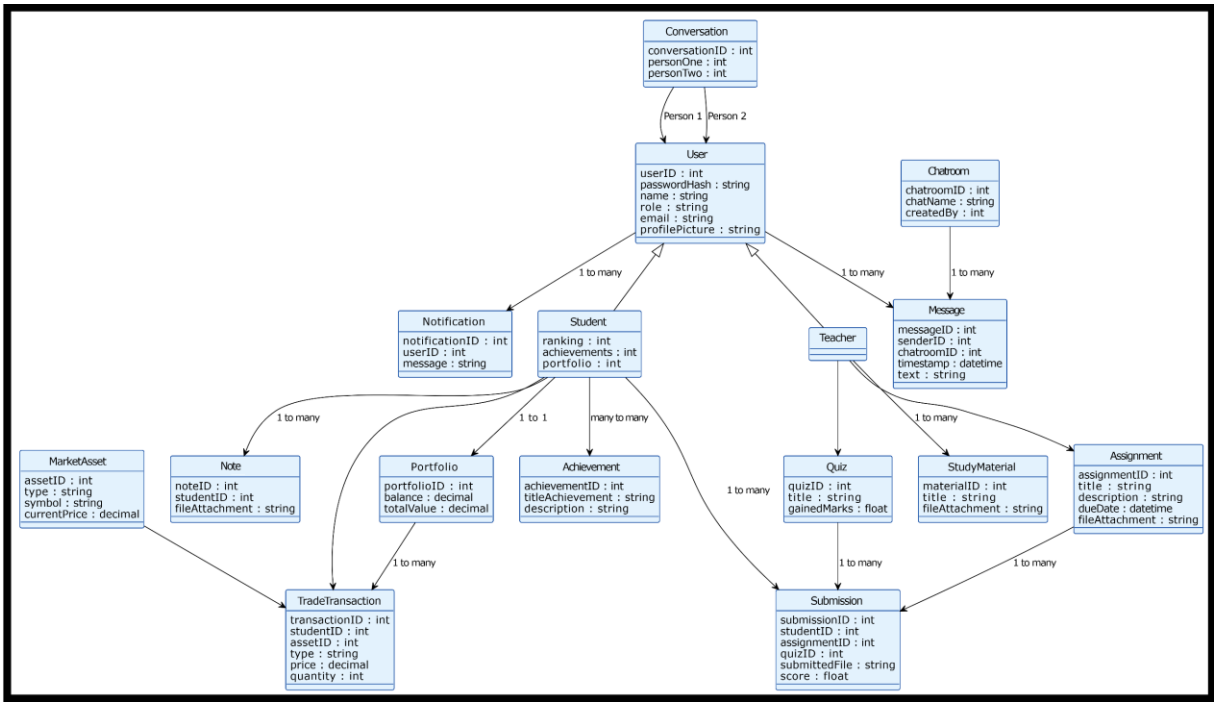
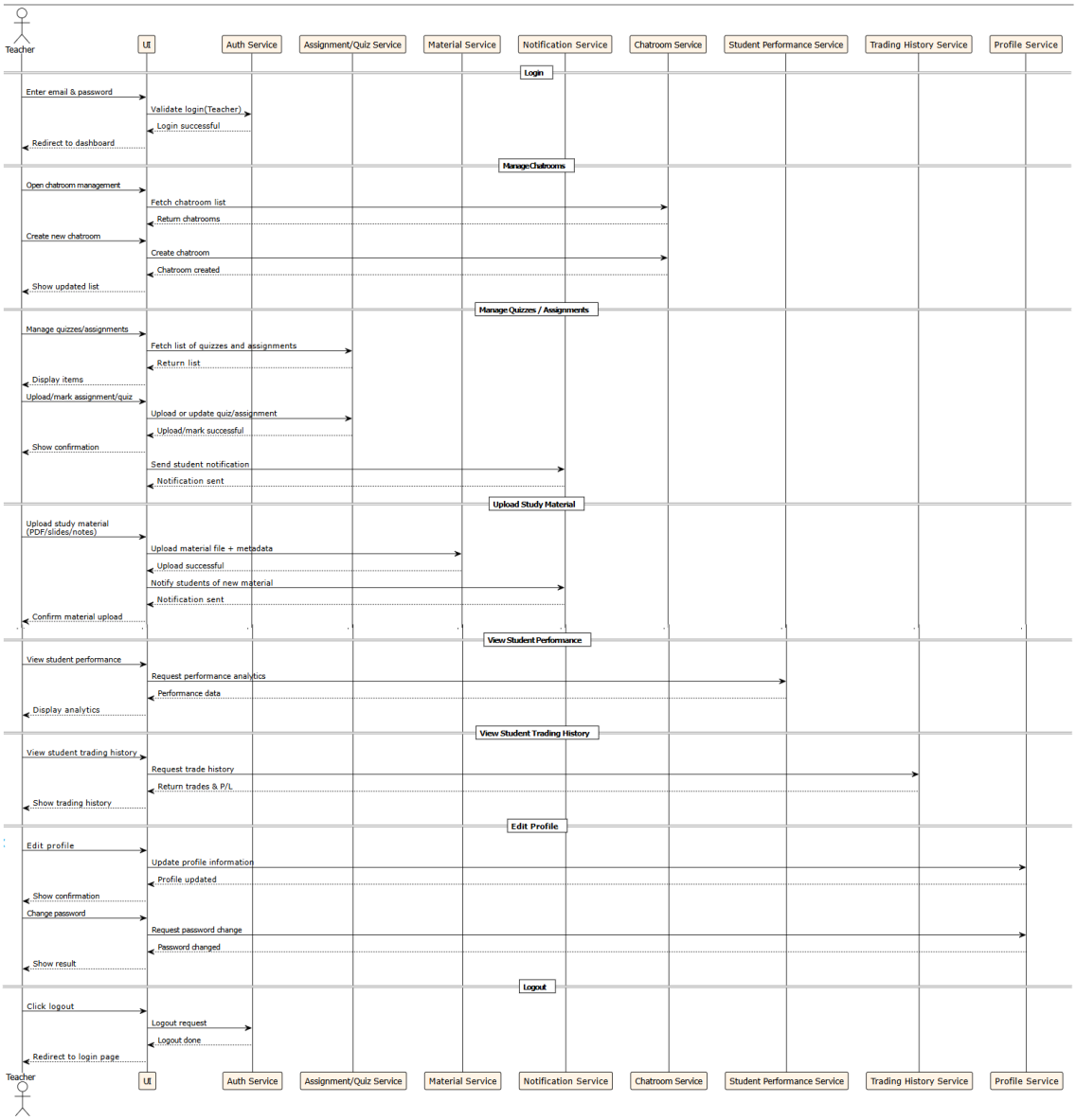


Figure 5.3.: Entity-Relationship Diagram

5.4 Interaction Viewpoint



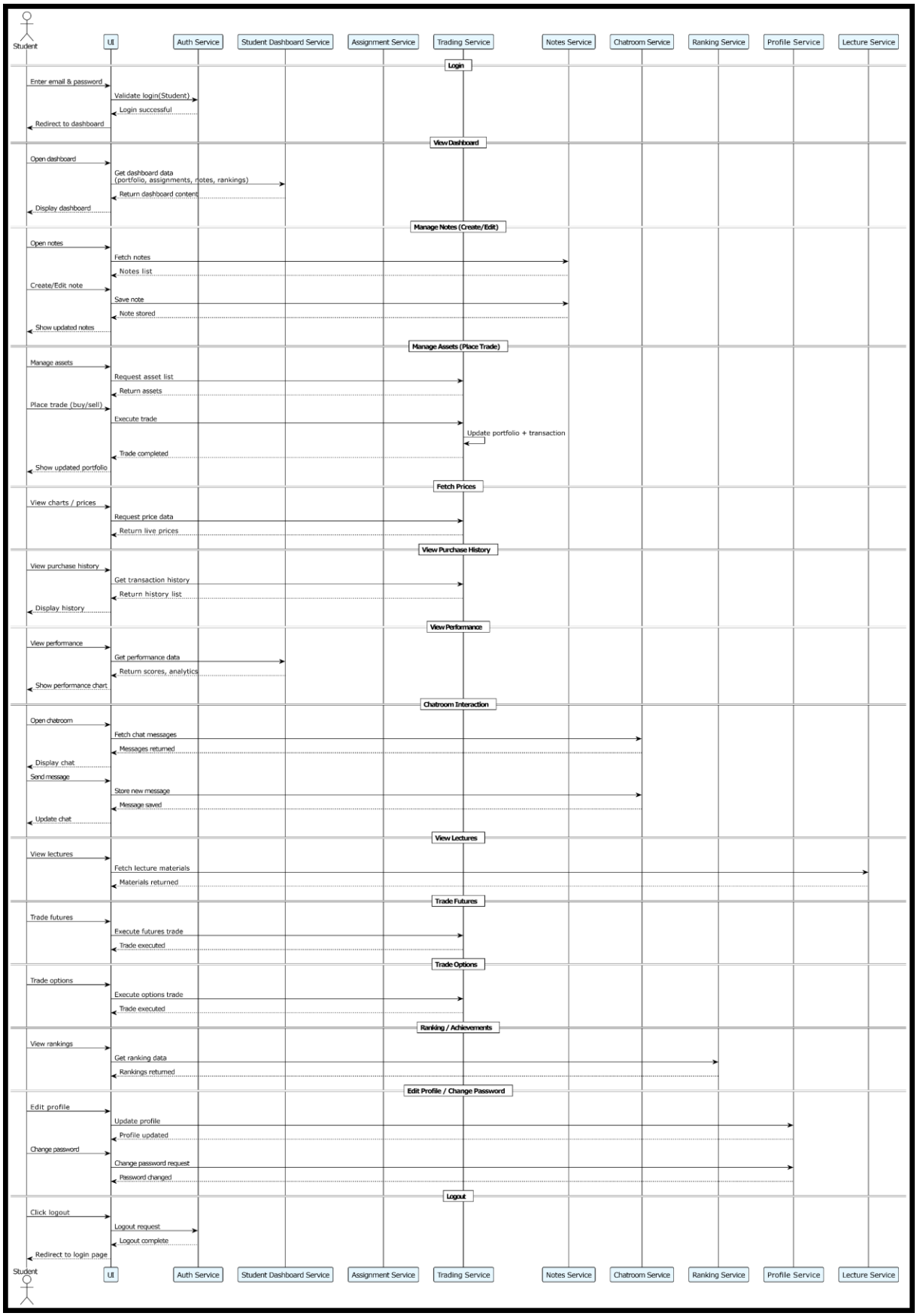


Figure 5.4.: Sequence Diagram

5.5 State Dynamics Viewpoint

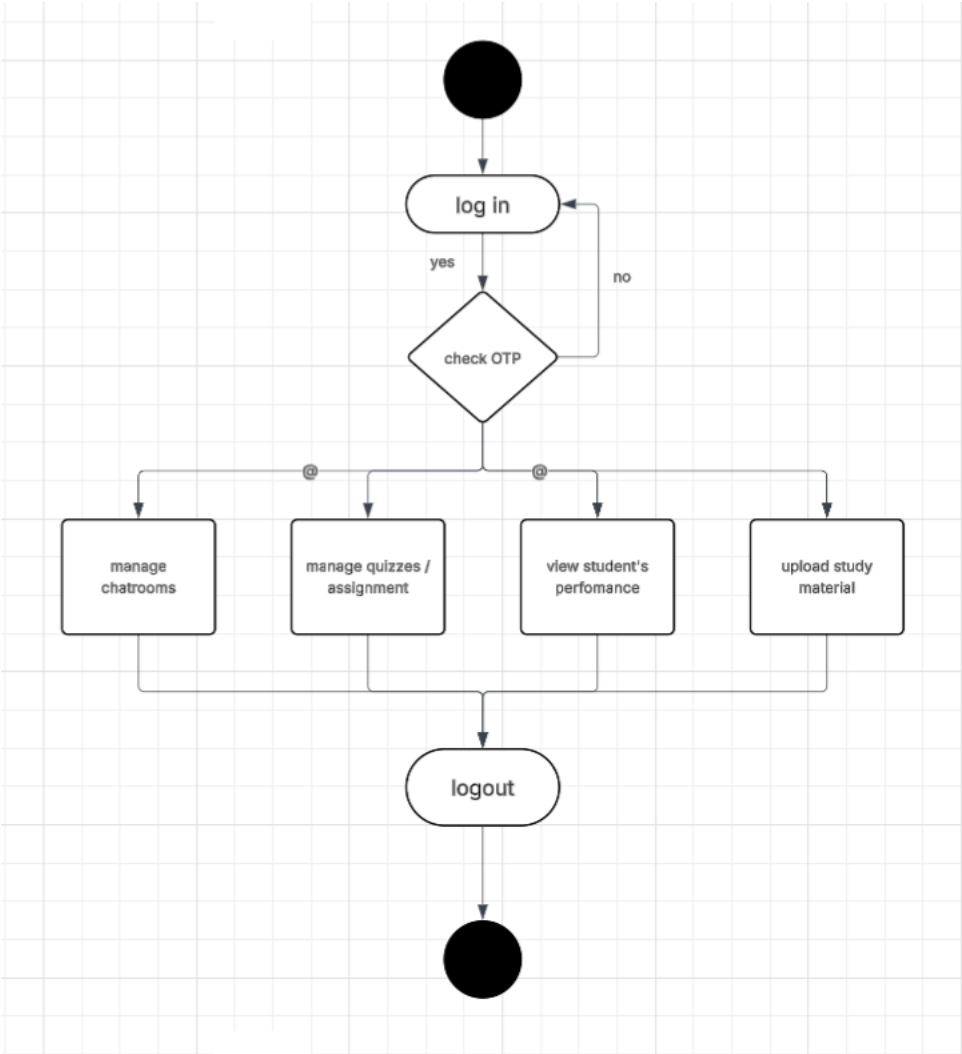


Figure 5.5.1: State Diagram for Teacher

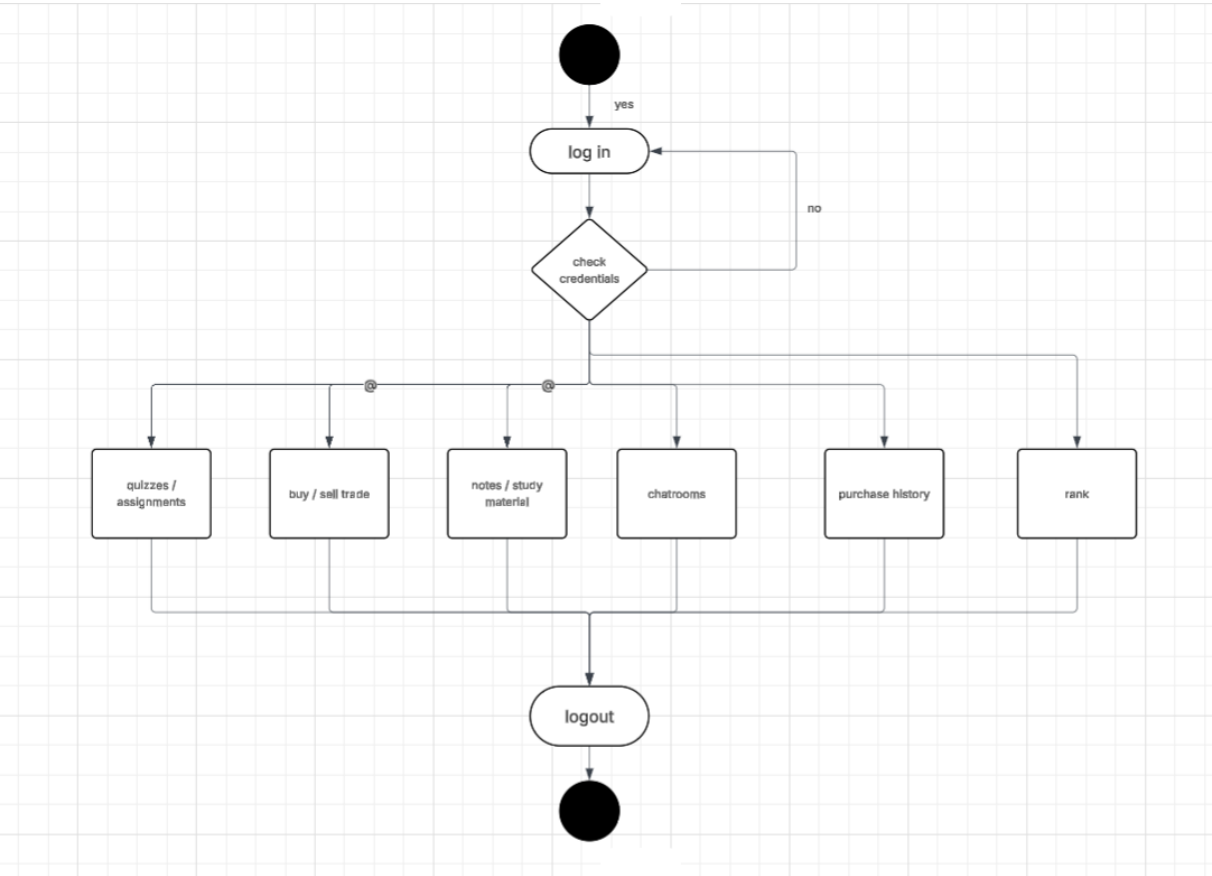


Figure 5.5.2: State Diagram for Student

5.6 Algorithm Viewpoint

5.6.1 Student Process:

1. Login Process

```
CLASS LoginPage:
   INITIALIZE navigate = useNavigate()
   INITIALIZE login = AuthContext.login
   INITIALIZE formData:
      email = ""
      password = ""

   INITIALIZE loading = false

   FUNCTION handleChange(event):
      field = event.target.name
      value = event.target.value
      UPDATE formData[field] = value

   FUNCTION handleSubmit(event):
      PREVENT default submit
```

```
        SET loading = true

        TRY:
            CALL login(formData.email, formData.password)
            NAVIGATE "/home"
        CATCH error:
            PRINT "Login error"
        FINALLY:
            SET loading = false

    RENDER UI:
        SHOW title "InterStock"
        SHOW subtitle "Hey, Login Now!"

        FORM onSubmit = handleSubmit:
            INPUT email
            INPUT password
            LINK "Forgot Password?"

            BUTTON:
                IF loading → LABEL "Logging in..."
                ELSE → LABEL "Log into your Account"

        SHOW "Or login with"
        DISPLAY Google Login Button
        SHOW "Not a Member yet?" with Sign-Up link
END CLASS
```

2. User Registration Process:

```
CLASS RegisterEmailPage:
    INITIALIZE navigate = useNavigate()

    INITIALIZE formData:
        name = ""
        email = ""
        confirmEmail = ""
        password = ""
        confirmPassword = ""

    INITIALIZE errors = {}
    INITIALIZE loading = false

    FUNCTION handleChange(event):
        field = event.target.name
        value = event.target.value
```

```
  UPDATE formData[field] = value

  IF errors[field] exists:
     CLEAR errors[field]

FUNCTION validateForm():
  INITIALIZE newErrors = {}

  IF name is empty:
     newErrors.name = "Name is required"

  IF email invalid or empty:
     newErrors.email = "Valid email required"

  IF confirmEmail empty OR email ≠ confirmEmail:
     newErrors.confirmEmail = "Emails do not match"

  IF password invalid or length < 8:
     newErrors.password = "Password too short"

  IF confirmPassword empty OR password ≠ confirmPassword:
     newErrors.confirmPassword = "Passwords do not match"

  UPDATE errors = newErrors
  RETURN TRUE if no errors else FALSE

FUNCTION handleSubmit(event):
  PREVENT default

  IF validateForm() == FALSE:
     EXIT

  SET loading = true

  TRY:
     PRINT "Registration data" + formData
     NAVIGATE "/otp-verify"
  FINALLY:
     SET loading = false
RENDER UI:
  SHOW Header
  SHOW form container with:
     Name
     Email
     Confirm Email
     Password
```

```
        Confirm Password

        BUTTON:
            IF loading → "Creating Account..."
            ELSE → "Create Account"

    SHOW "Already have an account?" with login link
END CLASS
```

3. Home Page Display Process:

```
CLASS HomePage:

  INITIALIZE navigate = useNavigate()

  INITIALIZE trendingStocks = [
    (AAPL, 175.50, +2.5),
    (GOOGL, 142.30, -1.2),
    (MSFT, 378.90, +3.1)
  ]


  RENDER UI:
    DISPLAY greeting "Hi, Ishmal Fatima"
    DISPLAY "Welcome to InterStock!"

    SHOW Notification and Utility Menu

    SHOW PortfolioOverview component

    BUTTON "Buy" → navigate("/buy-sell-stock")
    BUTTON "Sell" → navigate("/buy-sell-stock")

    SECTION "Trending Stocks":
        LOOP stock in trendingStocks:
            SHOW StockCard(stock)


    SHOW QuizHistory component
END CLASS
```

4. Learn Trading Page:

```
CLASS LearnPage:

  INITIALIZE navigate = useNavigate()
  INITIALIZE activeTab = "stocks"
```

```
    INITIALIZE searchQuery = ""

    INITIALIZE tabs = ["stocks", "futures", "options"]

    INITIALIZE stocks = [AAPL, GOOGL, MSFT, TSLA]


    RENDER UI:
        SHOW heading "Learn Trading"

        SEARCH BAR:
            INPUT searchQuery

        TABS SECTION:
            FOR each tab:
                SHOW button
                ON CLICK → activeTab = tab

        STOCK LIST:
            FOR each stock:
                SHOW StockCard(stock)
END CLASS
```

5.6.2 Profile Management Process:

5. Edit Profile Process:

```
CLASS EditProfilePage:

    INITIALIZE navigate = useNavigate()

    INITIALIZE formData:
        name = "Ishmal Fatima"
        email = "ishmalfatima@example.com"
        phone = "+92 334 5678900"

    INITIALIZE image = null


    FUNCTION handleChange(event):
        field = event.target.name
        value = event.target.value
        UPDATE formData[field] = value


    FUNCTION handleImageChange(event):
        file = event.target.files[0]
```

```
        IF file exists:
            reader = new FileReader()
            reader.onload = SET image = reader.result
            reader.readAsDataURL(file)


    FUNCTION handleSubmit(event):
        PREVENT default
        SAVE updated data (future backend)
        NAVIGATE "/profile"


    RENDER UI:
        HEADER:
            Back Button
            Title "Edit Profile"

        FORM:
            Profile Image Upload
            Full Name Input
            Email Input
            Phone Input

            SUBMIT BUTTON "Save Changes"
END CLASS
```

5.6.3 Additional Student Processes:

6. Profile Page Navigation:

```
CLASS ProfilePage:

    INITIALIZE navigate = useNavigate()
    INITIALIZE logout = AppContext.logout

    DEFINE profileItems = [
        Notification,
        Edit Profile,
        Change Password,
        My Notes,
        Privacy Policy,
        Terms & Conditions,
        Help Center,
        FAQs,
        Contact Us,
        Deactivate Account
    ]
```

```
    RENDER UI:
        SHOW user avatar + name + email

        LOOP item in profileItems:
            SHOW menu row (icon + title)
            IF item.isDanger:
                color red

            ON CLICK → item.action

        LOGOUT BUTTON → logout()
END CLASS
```

7. Ranking System Process:

```
CLASS RankPage:

    INITIALIZE activeTab = "global"
    INITIALIZE tabs = ["Class 1"]

    INITIALIZE participants = [
        (1, Jawad, 95),
        (2, Maha, 92),
        (3, You, 88),
        (4, Ayesha, 85),
        (5, Badar, 82)
    ]

    FUNCTION getRankIcon(rank):
        IF rank == 1 → Gold
        ELSE IF rank == 2 → Silver
        ELSE IF rank == 3 → Bronze
        ELSE → return "#rank"

    RENDER UI:
        SHOW "Rank"
         TABS:
            ON CLICK → change activeTab

        SHOW TOP 3 PODIUM

        PARTICIPANT LIST:
            FOR each participant:
                SHOW row with avatar + name + rating
                IF user:
```

```
            highlight row
END CLASS
```

Teacher Process:

1. Teacher Dashboard Process:

```
FUNCTION TeacherDashboard():

   IF NOT ValidateJWT(token):
      REDIRECT TO LoginPage
      RETURN
   END IF

   teacherID = GetUserIDFromJWT(token)

   DISPLAY TeacherDashboardHome()

   LOOP
      action = GetTeacherSelectedAction()

      SWITCH(action):
         CASE "UPLOAD_ASSIGNMENT":
            CALL UploadAssignment(teacherID)
         CASE "UPLOAD_QUIZ":
            CALL UploadQuiz(teacherID)
         CASE "UPLOAD_STUDY_MATERIAL":
            CALL UploadStudyMaterial(teacherID)
         CASE "CREATE_CHATROOM":
            CALL CreateChatroom(teacherID)
         CASE "MONITOR_CHATROOM":
            CALL MonitorChatroom(teacherID)
         CASE "CHECK_SUBMISSIONS":
            CALL CheckSubmissions(teacherID)
         CASE "REVIEW_TRADING_PERFORMANCE":
            CALL ReviewTradingPerformance(teacherID)
         CASE "SEND_ANNOUNCEMENT":
            CALL SendAnnouncement(teacherID)
         CASE "MARK_ASSIGNMENTS":
            CALL MarkAssignments(teacherID)
         CASE "MARK_QUIZZES":
            CALL MarkQuizzes(teacherID)
         CASE "LOGOUT":
            CALL Logout()
            EXIT LOOP
      END SWITCH
   END LOOP
```

```
END FUNCTION
```

2. Upload Assignment Process:

```
FUNCTION UploadAssignment(teacherID):
   assignmentData = GetAssignmentInput()
   IF ValidateAssignment(assignmentData):
      SaveAssignmentToDB(teacherID, assignmentData)
      DISPLAY "Assignment Uploaded Successfully"
   ELSE:
      DISPLAY "Invalid Assignment Details"
   END IF
END FUNCTION
```

Upload Quiz Process:

```
FUNCTION UploadQuiz(teacherID):
      quizData = GetQuizInput()
   IF ValidateQuiz(quizData):
      SaveQuizToDB(teacherID, quizData)
      DISPLAY "Quiz Uploaded Successfully"
   ELSE:
      DISPLAY "Invalid Quiz Data"
   END IF
END FUNCTION
```

6. Student Submission Review Process

6.1 Check Submissions

```
FUNCTION CheckSubmissions(teacherID):
   submissions = FetchStudentSubmissions(teacherID)
   DISPLAY submissions
END FUNCTION
```

Mark Quizzes:

```
FUNCTION MarkQuizzes(teacherID):
   quizzes = FetchPendingQuizAttempts()
   FOR each quiz IN quizzes:
      grade = GetTeacherInputGrade()
      SaveQuizGrade(quiz, grade)
   END FOR
   DISPLAY "Quizzes Marked Successfully"
END FUNCTION
```